# Última Dosis: motor de juego 2D por turnos con pathfinding

**Informe técnico del proyecto integrador · Temática A**

Universidad Cooperativa de Colombia · Facultad de Ingeniería · Ingeniería de Software

Curso: Estructuras de Datos (ID 706132) · Estudiante: Juan José Correa · Modalidad individual

Repositorio: github.com/correajuanjose200-coder/proyecto-ed-correajuanjose200-coder · Etiqueta de entrega: `entrega-final` (commit 41ed795) · Correcciones posteriores: `entrega-final-correcciones` (commit b93d17c)

Versión revisada del informe: 13 de noviembre de 2026 (la versión original se entregó el 11 de noviembre de 2026)

## 1. Resumen

Última Dosis es un juego por turnos en consola. Pedro debe cruzar una ciudad infectada hasta el aeropuerto mientras dos zombis lo persiguen por el camino más corto. El valor del proyecto no está en el juego, sino en las decisiones que lo sostienen: para cada capacidad del dominio (deshacer, inventario, eventos, rutas, mapa) se eligió una estructura de datos implementada desde cero, se declaró su operación dominante con su complejidad y se comparó con una alternativa descartada.

Se implementaron seis componentes (pila, cola, inventario, historial, mapa y rutas con BFS), integrados en un juego jugable con 126 pruebas automatizadas que pasan completas. No se implementaron el AVL, el montículo ni la cola de turnos propia, de modo que la funcionalidad F-08 (buscar entidades y ranking) queda sin cubrir. Este informe documenta con exactitud qué se construyó, qué no y por qué.

## 2. Descripción del sistema

Pedro empieza en la esquina superior izquierda (0, 0) y el aeropuerto (E) está en la esquina opuesta de una rejilla de 20 × 20 celdas. Al generar el mapa se abre primero un corredor libre que solo baja o avanza a la derecha, y después se colocan obstáculos fuera de él: el 20 % de las celdas (80 de 400), por debajo del máximo del 25 % que exige el

enunciado. La misma semilla produce siempre el mismo mapa. En el camino hay comida (C) y agua (A), que se recogen al pisarlas y se guardan en un inventario de hasta cinco objetos. La salud inicial es 100; la comida suma 20 y el agua suma 10, sin pasar de 100.

En cada turno Pedro puede moverse (w, a, s, d), esperar (x), usar un objeto (u N), soltar un objeto (g N), deshacer (z), rehacer (r), listar el inventario (i), pedir ayuda (h) o salir (q). Tras cada acción válida, cada zombi calcula una ruta hacia Pedro y avanza una casilla. Si un zombi ya está adyacente a Pedro cuando le toca moverse, lo ataca y le quita 25 puntos de salud. Hay victoria al llegar a la celda E y derrota si la salud llega a 0. Si ambas ocurren en el mismo turno, la victoria tiene prioridad (CA-28).

# 3. Estado real de la entrega

| Componente | Estado | Módulo |
|---|---|---|
| Pila | Implementada y probada | `estructuras/pila.py` |
| Cola | Implementada y probada | `estructuras/cola.py` |
| Inventario (lista enlazada) | Implementado y probado | `estructuras/inventario.py` |
| Historial (deshacer y rehacer) | Implementado y probado | `dominio/historial.py`, `dominio/accion.py` |
| Mapa (matriz densa, semilla) | Implementado y probado | `dominio/mapa.py` |
| Rutas (grafo implícito + BFS) | Implementadas y probadas | `dominio/rutas.py` |
| Juego y consola | Implementados; pruebas de integración parciales | `dominio/juego.py`, `main.py` |
| AVL (índice de entidades) | No implementado | n/a |
| Montículo (ranking) | No implementado | n/a |
| Cola de turnos propia | No implementada (orden fijo) | n/a |
| Antídoto (CA-05) y aturdimiento (CA-22) | No implementados | n/a |

La suite tiene 126 pruebas con pytest y pasa completa. El cambio de prioridad del AVL y el montículo, de Must a Won't, está registrado con su motivo en `specs/00-producto/moscow.md` versión 1.1.

# 4. Arquitectura

El sistema tiene tres capas. La capa de estructuras contiene las estructuras genéricas, sin conocimiento del juego. La capa de dominio contiene las reglas y usa las estructuras. La capa de consola solo lee comandos y dibuja.

| Capa | Módulo | Responsabilidad | Usa |
|---|---|---|---|
| Consola | `main.py` | Bucle de comandos, dibujo y mensajes | Juego |
| Dominio | `juego.py` | Reglas de turno, zombis, salud, victoria y derrota | Mapa, Rutas, Historial, Inventario, Cola |
| Dominio | `rutas.py` | Camino más corto (BFS) y primer paso | Mapa, Cola |
| Dominio | `historial.py`, `accion.py` | Registro de acciones, deshacer y rehacer | Pila |
| Dominio | `mapa.py` | Rejilla, generación por semilla, dibujo | n/a |
| Estructuras | `pila.py`, `cola.py` | LIFO y FIFO sobre nodos enlazados | Nodo |
| Estructuras | `inventario.py` | Lista enlazada con puntero al último nodo | Nodo |
| Estructuras | `nodo.py` | Nodo enlazado simple | n/a |

**Flujo de un turno.** Se describe `mover(tecla)` en `juego.py`. (1) Se rechaza la acción si la partida terminó, si la tecla no es una dirección, si la celda es un obstáculo o el borde, o si hay un zombi encima. (2) Si la celda tiene un objeto y el inventario no está lleno, se agrega al inventario, se limpia la celda del mapa, se emite el evento «Recogiste…» y se llama a `_turno("recoger", …)` pasando el objeto y su posición en el inventario. Si el inventario está lleno, se emite un evento y el objeto queda en el suelo. (3) `_turno` guarda el estado de antes (Pedro, zombis y contador de turnos), actualiza a Pedro, incrementa el turno y llama a `_mover_zombis`. (4) `_mover_zombis` recorre cada zombi y calcula `buscar_ruta` hacia Pedro: si no hay ruta, el zombi no se mueve; si está a distancia 1 o menos, ataca (−25 de salud y evento de colisión); si no, avanza al primer paso de la ruta, salvo que esa celda la ocupe otro zombi. (5) Por último, `_turno` registra en el `Historial` una `Accion` con el estado de antes y de después y el objeto afectado. Los eventos quedan en una `Cola` y la consola los muestra una sola vez, en orden de llegada.

**Decisión de diseño.** `main()` solo imprime el encabezado; el bucle interactivo corre únicamente al ejecutar `python main.py`. Así la prueba de arranque de pytest no queda esperando una entrada.

# 5. Aplicación de Spec-Driven Development

El flujo fue: historia de usuario, priorización MoSCoW, `spec.md` (contrato con criterios CA), `plan.md` (estructura y complejidad), `tasks.md` y, por último, pruebas y código. Cada prueba nombra en su docstring el criterio que verifica, y `specs/trazabilidad.md` (generada a partir de esos docstrings) permite navegar criterio, prueba y código en ambas direcciones.

El historial de Git muestra el orden esperado para los componentes de rutas: `spec(rutas)`, luego `test(rutas)` y luego `feat(rutas)`.

## 5.1 Cambios de especificación y su motivo

| Documento | Cambio | Motivo |
|---|---|---|
| Spec del historial v1.1 | CA-H2 se hizo coherente con las reglas del historial | Contradicción detectada al escribir las pruebas de deshacer y rehacer |
| Plan del historial v1.1 | Se declaran las tuplas como soporte de `Accion` | Regla 4: todo soporte interno debe justificarse |
| `moscow.md` v1.1 | AVL y montículo pasan de Must a Won't | Falta de tiempo; se priorizó un juego jugable con pruebas |
| `estructuras.md` v1.1 | Se registra que no hay cola de turnos propia, ni AVL ni montículo | Coherencia con lo implementado |
| `99-sistema/spec.md` v1.1 | Componentes reales, reglas y lista de lo no implementado | Cierre de la entrega final |
| `99-sistema/spec.md` v1.2 | Comparación del contrato del enunciado con lo construido | Preparar la sustentación; no cambia comportamiento |

### 5.2 Un caso de especificación viva

Durante la integración se detectó que el primer borrador de `juego.py` usaba valores distintos a la spec (salud 3 en lugar de 100, comida +2 en lugar de +20, y la derrota evaluada antes que la victoria). Se corrigió el código para que coincidiera con la spec y se ajustaron las pruebas, en lugar de cambiar la especificación. La spec fue la fuente de verdad.

### 5.3 Autocrítica sobre SDD

La disciplina de escribir la spec antes del código se sostuvo bien en historial, pila, mapa y rutas. En los últimos días del plazo, la integración del juego y los documentos de cierre se concentraron, y varios criterios quedaron implementados sin una prueba dedicada. La lección es que la trazabilidad debe mantenerse al día con cada componente y no reconstruirse al final.

## 6. Las estructuras de datos

Para cada estructura se indica qué es, cómo se implementó, dónde se usa, su complejidad y la alternativa descartada.

### 6.1 Pila: dos pilas para deshacer y rehacer

**Qué es.** Una estructura LIFO: el último elemento que entra es el primero que sale.

**Implementación.** Nodos enlazados con un puntero a la cima, sin usar `list`. Operaciones: apilar, desapilar, cima, tamaño, esta_vacia y vaciar. Desapilar o leer la cima de una pila vacía lanza `PilaVaciaError`.

**Dónde se usa.** `Historial` mantiene dos pilas: *hechas* (acciones que se pueden deshacer) y *deshechas* (acciones que se pueden rehacer). Registrar una acción nueva apila en hechas y vacía deshechas (CA-17). Deshacer desapila de hechas y apila en deshechas; rehacer hace lo contrario. Si no hay nada que deshacer o rehacer, el historial devuelve `None` sin cambiar el estado (CA-18).

**Por qué esta estructura.** Deshacer actúa siempre sobre la acción más reciente, que es exactamente el comportamiento de una pila.

**Complejidad.** Apilar, desapilar, cima, tamaño y esta_vacia son O(1). Vaciar recorre los nodos, O(n). Por eso registrar una acción nueva es O(1) amortizado: cada acción se descarta de la pila de deshechas una sola vez.

**Alternativa descartada.** Una lista doble con cursor también da O(1), pero obliga a mantener el cursor y a cortar la lista cada vez que se registra una acción nueva; es más código para el mismo resultado.

### 6.2 Cola: eventos y frontera del BFS

**Qué es.** Una estructura FIFO: el primero que entra es el primero que sale.

**Implementación.** Nodos enlazados con punteros al frente y al final, de modo que encolar y desencolar no recorren la estructura. Desencolar o consultar el frente de una cola vacía lanza `ColaVaciaError`.

**Dónde se usa.** En dos lugares con la misma necesidad de orden de llegada. Primero, los eventos del juego salen en el mismo orden en que ocurrieron y cada uno se muestra una sola vez (CA-23 y CA-25). Segundo, es la frontera del recorrido en anchura de `rutas.py`.

**Complejidad.** Encolar, desencolar, frente, tamaño y esta_vacia son O(1).

**Alternativa descartada.** Una cola de prioridad: todos los eventos tienen la misma urgencia, así que la prioridad solo añadiría O(log n) sin beneficio.

### 6.3 Inventario: lista enlazada simple

**Qué es.** Una secuencia ordenada de objetos con capacidad máxima de cinco.

**Implementación.** Lista enlazada simple con referencia al último nodo. Agregar inserta al final; insertar y quitar trabajan por posición (desde 0) y listar recorre en orden. Agregar con el inventario lleno lanza `InventarioLlenoError`, y una posición inexistente lanza `PosicionInventarioError`.

**Dónde se usa.** Recoger un objeto lo agrega al final y conserva el orden de recogida. Usar y soltar lo quitan por posición. Como el inventario no conoce los efectos de los objetos, deshacer puede reinsertar un objeto en su posición original porque `Accion` guarda el símbolo y la posición. Si el inventario está lleno, el objeto se queda en el mapa y se genera un evento (CA-12).

**Complejidad.** Agregar es O(1) gracias al puntero al último nodo. Insertar, quitar y listar son O(n), con n como máximo 5, por lo que el costo es despreciable.

**Alternativa descartada.** Un arreglo de capacidad 5. Con ese tamaño el costo sería igual, pero quitar del medio obliga a desplazar elementos y a manejar posiciones vacías. Con una capacidad grande, el arreglo ganaría en acceso por posición, O(1), a cambio de desplazar en O(n).

## 6.4 Mapa: matriz densa

**Qué es.** Una rejilla de filas por columnas con los símbolos libre (.), obstáculo (#), meta (E), comida (C) y agua (A).

**Implementación.** Las operaciones obtener, poner, dentro y es_transitable son accesos directos por índice. La generación recibe una semilla: la misma semilla produce el mismo mapa, abre primero un corredor libre que solo baja o avanza a la derecha entre el inicio (0, 0) y la meta, y coloca después el 20 % de las celdas como obstáculos, fuera de ese corredor (CA-01 y CA-02).

**Complejidad.** Consultar o cambiar una celda es O(1); contar, dibujar y calcular la proporción de obstáculos son O(F × C).

**Soporte interno.** La matriz usa `list` solo como contenedor de tamaño fijo; todas las operaciones están implementadas en `Mapa`.

**Alternativa descartada.** Una matriz dispersa solo ahorra memoria si casi todo está vacío. Con 400 celdas y hasta 25 % de obstáculos, añadiría una búsqueda en cada consulta.

## 6.5 Grafo implícito y búsqueda en anchura (BFS)

**Qué es.** Cada celda transitable es un vértice y cada par de celdas vecinas transitables es una arista. El grafo no se construye: los vecinos (arriba, abajo, izquierda y derecha, en orden fijo) se calculan al vuelo con `dentro` y `es_transitable`. Esto ahorra memoria y evita mantener aristas cuando el mapa cambia.

**Implementación.** `buscar_ruta(mapa, origen, destino)` usa la `Cola` propia como frontera y una matriz de padres. Una celda con valor `None` aún no se ha descubierto; el valor guardado es la celda desde la que se llegó. Al descubrir el destino se camina hacia atrás por los padres contando pasos, y la última celda antes del origen es el primer paso. Devuelve distancia, siguiente y exploradas; devuelve `None` si no hay camino, lanza `PosicionInvalidaError` si un extremo está fuera del mapa y no modifica el mapa (CA-R1 a CA-R9, CA-19, CA-20, CA-21).

**Por qué da el camino más corto.** La cola es FIFO, así que el recorrido avanza por capas: todas las celdas a distancia k se procesan antes que las de distancia k + 1. La primera vez que se descubre una celda es, por tanto, por un camino de longitud mínima. El orden fijo de vecinos hace el resultado determinista incluso con empates.

**Complejidad.** O(V + E), que en la rejilla equivale a O(F × C) por zombi y por turno. Con dos zombis, un turno cuesta O(Z × F × C).

**Soporte interno.** La matriz de padres usa `list` y se declara en el plan.

**Alternativa descartada.** A*. Con una heurística admisible (la distancia de Manhattan lo es en una rejilla de cuatro vecinos) exploraría menos celdas, pero con unas 400 celdas la diferencia práctica es pequeña, y BFS es más simple de probar y de explicar. El campo `exploradas` ya permite medir la comparación cuando se implemente.

### 6.6 Capacidades sin estructura propia: diseño no implementado

Tres capacidades del dominio no quedaron resueltas con una estructura propia. Se documenta qué se habría elegido y por qué, para que la decisión sea explícita.

| Capacidad | Estructura prevista | Operación dominante | Alternativa | Estado |
|---|---|---|---|---|
| Índice de entidades (F-08) | AVL | Buscar por identificador, O(log n) garantizado | BST simple: O(n) en el peor caso si las inserciones llegan ordenadas | No implementado |
| Ranking de puntajes | Montículo | Obtener el máximo en O(1), insertar en O(log n) | Lista ordenada: insertar O(n) | No implementado |
| Orden de turnos | Cola o lista circular | Siguiente turno en O(1) | Orden fijo en el código | Orden fijo, sin estructura propia |

El orden de turnos funciona hoy porque hay un jugador y dos zombis con un ciclo fijo, pero no es una estructura declarada y no se defiende como tal.

## 7. Análisis de complejidad

| Módulo | Operación | Complejidad |
|---|---|---|
| pila | apilar, desapilar, cima, tamaño, esta_vacia | O(1) |
| pila y cola | vaciar | O(n) |

| Módulo | Operación | Complejidad |
|---|---|---|
| cola | encolar, desencolar, frente, tamaño, esta_vacia | O(1) |
| inventario | agregar | O(1) |
| inventario | insertar, quitar, listar | O(n), con n ⩽ 5 |
| historial | registrar | O(1) amortizado |
| historial | deshacer, rehacer | O(1) |
| mapa | obtener, poner, dentro, es_transitable | O(1) |
| mapa | contar, dibujar, proporción de obstáculos | O(F × C) |
| rutas | buscar_ruta (BFS) | O(V + E) = O(F × C) |
| juego | un turno | O(Z × F × C) |
| juego | deshacer y rehacer | O(1) en el historial, más O(n) del inventario si hay objeto |

La operación que domina el costo del juego es la búsqueda de rutas: un BFS por zombi en cada turno. Con un mapa de 20 × 20 y dos zombis son como máximo unos 800 vértices visitados por turno, un costo que no afecta la jugabilidad. Todo lo demás es constante o depende de un n máximo de 5.

## 7.1 Medición empírica: celdas exploradas por BFS

Camino de inicio a meta en mapas de 20 × 20 generados con las semillas 1 a 10 (320 celdas transitables en cada uno):

| Semilla | Pasos del camino | Celdas exploradas |
|---|---|---|
| 1 | 38 | 316 |
| 2 | 38 | 320 |
| 3 | 38 | 319 |
| 4 | 38 | 319 |

| Semilla | Pasos del camino | Celdas exploradas |
|---|---|---|
| 5 | 38 | 320 |
| 6 | 38 | 318 |
| 7 | 38 | 318 |
| 8 | 38 | 318 |
| 9 | 38 | 315 |
| 10 | 38 | 316 |

En todas las semillas el camino mide 38 pasos porque el corredor garantizado, que solo baja o avanza a la derecha, deja siempre una ruta de distancia Manhattan (19 + 19) entre esquinas opuestas. Lo que varía es el número de celdas exploradas. Este recorrido de esquina a esquina es un caso adverso para BFS, que visita casi todo el mapa, y justamente ahí A* podría explorar menos celdas. No se implementó A*, por lo que no hay comparación medida contra él.

## 8. Pruebas y trazabilidad

La suite tiene 126 pruebas con pytest y pasa completa. Hay pruebas por estructura (pila, cola, inventario, historial, acción, mapa, generación de mapa y rutas) y pruebas de integración en `tests/test_juego.py`. Cubren los extremos: estructuras vacías, un solo elemento, posiciones inexistentes, mapas de una celda, destinos encerrados y orígenes sobre obstáculos. Cada prueba indica en su docstring el criterio que verifica, y `specs/trazabilidad.md` (versión 2.0) se generó a partir de esos docstrings.

Las pruebas de integración verifican, entre otros: CA-15 (deshacer una recogida devuelve el objeto al mapa y lo quita del inventario), CA-16 (rehacer), CA-17 (una acción nueva vacía lo rehacible), CA-07 (los obstáculos bloquean y no gastan turno), CA-10 (usar comida suma 20), CA-23 (eventos en orden), y CA-26 y CA-27 (victoria y derrota).

### 8.1 Cobertura incompleta, declarada

Según la trazabilidad vigente:

- **Implementados, sin prueba que los nombre:** CA-03, CA-06, CA-08, CA-24, CA-25, CA-28 y CA-36.

- **Parciales:** CA-22 (existen el bloqueo y el daño de -25, pero no el aturdimiento) y CA-29 (mover, esperar, usar y soltar se rechazan al terminar la partida, pero deshacer y rehacer no comprueban el estado; la consola no llega a ellos).
- **No implementados:** CA-05 (antídoto) y CA-30 a CA-34 (AVL y montículo).

Esta lista reemplaza a la que traía la versión original del informe, que enumeraba más criterios como sin cubrir. La trazabilidad regenerada a partir de los docstrings es la fuente más reciente.

## 9. Dificultades y decisiones

**Gestión del tiempo.** La integración del juego, las estructuras de cola, inventario y rutas, y los documentos de cierre se concentraron en los últimos días del plazo. Por eso no se alcanzó a construir el AVL ni el montículo, y la cola de turnos se resolvió con un orden fijo.

**Deshacer los efectos de una acción.** No basta con restaurar la posición de Pedro: recoger, usar y soltar cambian el mapa y el inventario. Se resolvió guardando en cada `Accion` el estado de antes y de después, más el objeto afectado con su posición en el inventario.

**Archivos vacíos al pegar código.** Al pegar archivos largos en el editor del entorno, algunos quedaron vacíos y las pruebas fallaron por errores de importación. Se resolvió creando los archivos desde la terminal y verificando con `wc -l` antes de cada commit.

**Permisos de GitHub.** El Codespace estaba autenticado con una cuenta distinta a la dueña del repositorio, lo que impidió subir el commit del informe y las correcciones a tiempo. Se resolvió agregando esa cuenta como colaboradora y se documentó la situación ante el docente.

## 10. Declaración de uso de inteligencia artificial

Se usó un asistente de inteligencia artificial durante el proyecto, de la siguiente manera:

- **Entrega 1:** ayudó a redactar el borrador de los artefactos de especificación a partir del documento de diseño, que luego revisé y ajusté.
- **Entrega final:** ayudó a escribir el código de `rutas.py`, `juego.py` y `main.py`, las pruebas de `test_juego.py` y los borradores de `ANALISIS.md`, de la trazabilidad y de este informe.
- **Aclaración posterior (13 de noviembre):** el asistente también redactó borradores de las specs, las pruebas y el código de `cola` e `inventario`, y de la spec y las pruebas de `rutas`.

Todo código propuesto se comparó contra las especificaciones, se ejecutó con la suite de pruebas y se corrigió donde no coincidía. Soy responsable de poder explicar cada fragmento en la sustentación, incluyendo el flujo de `_turno`, el funcionamiento de deshacer y rehacer y el recorrido de `buscar_ruta`. El detalle está en `BITACORA.md`.

## 11. Conclusiones y trabajo futuro

El proyecto cumple el núcleo jugable: un mapa de 20 × 20 generado por semilla con camino garantizado, un jugador por turnos, dos enemigos que lo persiguen con el camino más corto, objetos, inventario, historial con deshacer y rehacer que restaura también los efectos, eventos en orden y condiciones de victoria y derrota, todo desde una consola.

Cada estructura implementada resuelve una necesidad real del sistema. Las pilas encajan con deshacer porque este siempre actúa sobre lo más reciente; la cola sirve tanto a los eventos como al recorrido en anchura, dos usos con la misma necesidad de orden de llegada; la lista enlazada conserva el orden del inventario; y el BFS sobre un grafo implícito da el camino más corto sin construir aristas.

Quedaron fuera el AVL, el montículo, la cola de turnos propia, el antídoto y el aturdimiento, por lo que F-08 no está disponible. Esto afecta la diversidad de estructuras demostradas, y es la principal brecha frente al enunciado.

**Lecciones.** La primera es integrar cada estructura al sistema apenas se termina, en lugar de concentrar el trabajo al final. La segunda es mantener la trazabilidad al día durante el desarrollo, no reconstruirla al cierre. La tercera es calcular la capacidad real con margen y recortar la versión de un Must antes de bajarlo de prioridad.

**Trabajo futuro.** Implementar el AVL de entidades y el montículo de ranking, A* con una comparación medida de celdas exploradas frente a BFS, la cola de turnos propia, y completar las pruebas de integración pendientes.

## Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Krekel, H., y colaboradores de pytest. (2024). *pytest documentation*. https://docs.pytest.org/

Python Software Foundation. (2024). *The Python language reference*. https://docs.python.org/3/reference/

Sedgewick, R., y Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley.

Universidad Cooperativa de Colombia. (2026). *Proyecto integrador, Temática A: Motor de juego 2D con pathfinding* [Documento técnico del curso Estructuras de Datos, ID 706132].